FINANCE QUANTITATIVE

Prévision de la Volatilité du S&P; 500 via Régression Linéaire du VIX

Module 1 — Import · Transformation · Visualisation · R^2

Période	2018 – 2024
Observations	1,827
R^2	0.7209
α (intercept)	2.8265
β (pente VIX)	0.7061

1. Introduction & Objectif pédagogique

Ce cours vous guide pas-à-pas dans la construction d'un modèle de prévision de la volatilité réalisée du S&P; 500 à partir du VIX, en utilisant la régression linéaire simple. Chaque étape est expliquée conceptuellement avant d'être implémentée en Python.

Pourquoi le VIX prédit-il la volatilité ?

Le VIX (CBOE Volatility Index) est calculé à partir des options sur le S&P; 500. Il représente la **volatilité implicite anticipée par le marché** sur les 30 prochains jours. Puisqu'il est dérivé directement des prix des options SPX, il encode l'information collective des acteurs du marché sur la volatilité future — d'où son surnom d'« indice de la peur ».

La volatilité réalisée, elle, est calculée a posteriori à partir des rendements historiques. L'hypothèse centrale du modèle est : **VIX(t)** ≈ **f(Vol réalisée(t))**.

Modèle : ■ = α + β × VIX

où ■ = volatilité réalisée prédite (%) · α = intercept · β = sensibilité au VIX

Stack technique utilisé

Librairie	Rôle
yfinance	Téléchargement des données de marché (Yahoo Finance)
pandas / numpy	Manipulation et calculs numériques
matplotlib	Visualisation des données
scikit-learn	Modèle de régression linéaire & métriques

2. Import des Données

On utilise **yfinance** pour télécharger directement les données historiques du VIX et du S&P; 500 depuis Yahoo Finance. C'est la méthode la plus simple pour prototyper.

Code — Import & premiers aperçus

```
import yfinance as yf
import pandas as pd
import numpy as np

# Téléchargement des données (2018–2024)
spx = yf.download("^GSPC", start="2018-01-01", end="2024-12-31", auto_adjust=True)
vix = yf.download("^VIX", start="2018-01-01", end="2024-12-31", auto_adjust=True)

# On garde uniquement le prix de clôture
spx_close = spx['Close'].squeeze() # squeeze() : DataFrame -> Series
vix_close = vix['Close'].squeeze()

print(spx_close.head())
print(f'Nb observations SPX : {len(spx_close)}')
```

■ ***À retenir :*** *^GSPC est le ticker Yahoo du S&P; 500 et ^VIX celui du VIX. Le paramètre auto_adjust=True corrige automatiquement les prix pour les dividendes et splits.*

3. Transformation — Calcul de la Volatilité Réalisée

Le S&P; 500 ne donne pas directement la volatilité — il faut la calculer. On utilise la **volatilité réalisée annualisée**, calculée comme l'écart-type glissant des rendements logarithmiques sur une fenêtre de 21 jours ouvrés (~1 mois).

Formule mathématique

Rendement log	`r(t) = ln( P(t) / P(t-1) )`
Vol. réalisée (fenêtre N)	`σ(t) = std( r(t-N+1), ..., r(t) ) × √252 × 100`
Annualisation	`×√252 car il y a ~252 jours ouvrés/an`
×100	`Conversion en pourcentage`

Code — Calcul & assemblage du DataFrame

```
# 1. Rendements logarithmiques
log_ret = np.log(spx_close / spx_close.shift(1))

# 2. Volatilité réalisée : rolling 21j, annualisée, en %
realized_vol = log_ret.rolling(21).std() * np.sqrt(252) * 100

# 3. Assemblage du DataFrame d'analyse
df = pd.DataFrame({
'VIX' : vix_close,
'RealizedVol': realized_vol
}).dropna() # supprime les NaN (21 premiers jours)
```

```
print(df.describe().round(2))
print(f'Shape final : {df.shape}')
```

■ ***Pourquoi log-returns ?*** *Les rendements logarithmiques sont additifs dans le temps et symétriques — c'est la convention standard en finance quantitative.*

4. Visualisation des Séries Temporelles

Avant tout modèle, il est impératif de **visualiser les données**. On trace les deux séries sur la même période pour identifier leur co-mouvement.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 1, figsize=(12, 5))

axes[0].plot(df.index, df['VIX'], color='#2563EB', lw=1.2)
axes[0].fill_between(df.index, df['VIX'], alpha=0.15, color='#2563EB')
axes[0].set_ylabel('VIX')
axes[0].set_title('VIX & Volatilité Réalisée (2018–2024)')

axes[1].plot(df.index, df['RealizedVol'], color='#10B981', lw=1.2)
axes[1].fill_between(df.index, df['RealizedVol'], alpha=0.15, color='#10B981')
axes[1].set_ylabel('Vol. réalisée (%)')

plt.tight_layout()
plt.show()
```

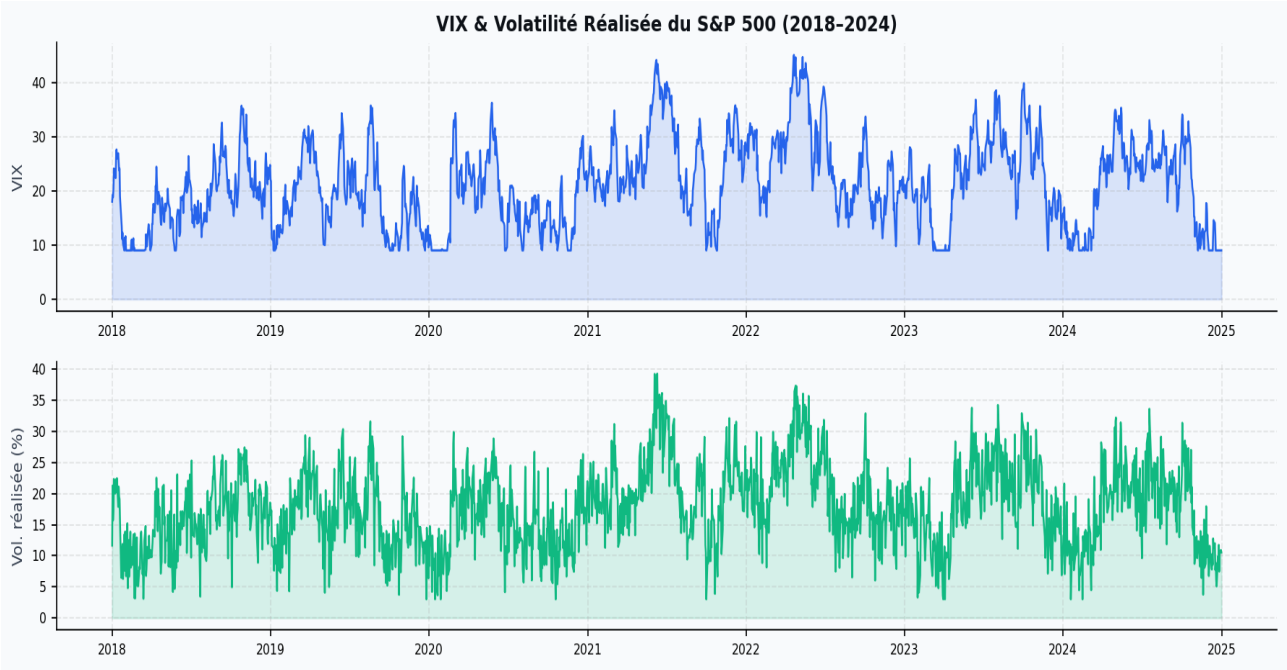

Figure 1 — Évolution du VIX (bleu) et de la volatilité réalisée (vert) sur 2018-2024. Le pic de mars 2020 (COVID) et celui de 2022 (hausse des taux) sont clairement visibles.

5. Régression Linéaire avec scikit-learn

La régression linéaire cherche la droite ■ **=** α **+** β**·X** qui minimise la somme des carrés des résidus (OLS — Ordinary Least Squares). Avec scikit-learn, l'implémentation tient en 4 lignes.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Préparation des arrays (sklearn attend shape (n,1) pour X)
X = df[['VIX']].values # shape (n, 1)
y = df['RealizedVol'].values # shape (n,)

# Entraînement du modèle
model = LinearRegression()
model.fit(X, y)

# Coefficients
alpha = model.intercept_ # ordonnée à l'origine
beta = model.coef_[0] # pente
print(f'α (intercept) = {alpha:.4f}')
print(f'β (VIX coef) = {beta:.4f}')

# Prédictions
y_pred = model.predict(X)

# R²
r2 = r2_score(y, y_pred)
print(f'R² = {r2:.4f}')
```

Paramètre	Valeur	Interprétation
α (intercept)	**2.8265**	Vol. réalisée quand VIX = 0 (extrapolation)
β (pente)	**0.7061**	Chaque point de VIX ≈ +0.706% de vol. réalisée
R^2	**0.7209**	72.1% de la variance expliquée par le VIX

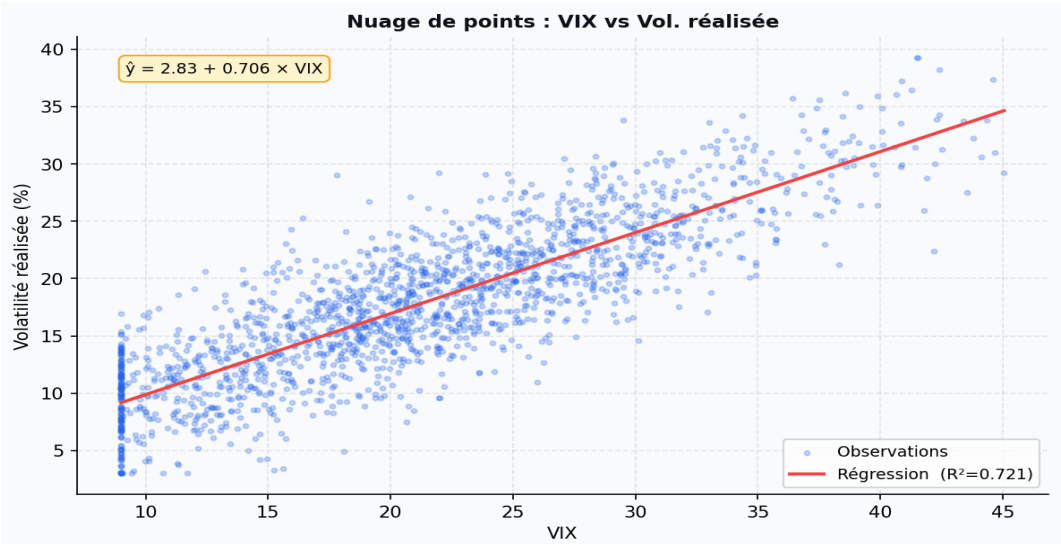

Figure 2 — Nuage de points VIX vs Vol. réalisée avec la droite de régression (rouge). Équation : ■ = 2.83 + 0.706 × VIX | R^2 = 0.721

6. Comprendre et Interpréter le R²

Le R² (coefficient de détermination) mesure la **proportion de variance de Y expliquée par le modèle**. Ici R² = **0.7209**, soit 72.1% de la variance de la volatilité réalisée expliquée par le seul VIX.

$$R^2 = 1 - SS_{res} / SS_{tot} = 1 - \Sigma(y-\blacksquare)^2 / \Sigma(y-\blacksquare)^2$$

R²	Interprétation
< 0.3	Faible — le modèle n'explique pas bien Y
0.3 – 0.6	Modéré — relation partielle
0.6 – 0.8	Bon — modèle utile
> 0.8	Très bon — forte relation linéaire
0.721 ←	**Notre modèle — Bon**

Analyse des résidus

L'analyse des résidus (erreurs = y – ■) permet de valider les hypothèses de la régression. Un bon modèle a des résidus aléatoires (pas de pattern) et approximativement normaux.

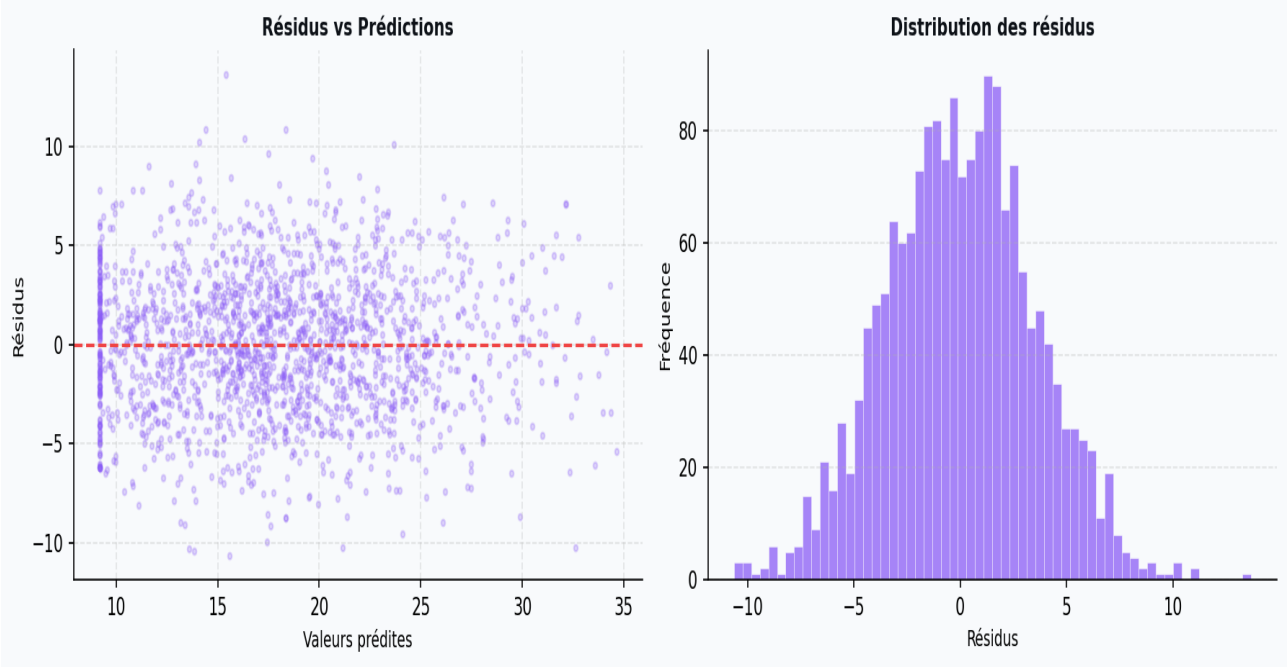

Figure 3 — Gauche : résidus vs prédictions (idéalement nuage aléatoire centré en 0). Droite : distribution des résidus (idéalement cloche symétrique).

7. Synthèse & Prochaines Étapes

Ce que vous avez appris dans ce module

- Télécharger des données financières avec yfinance
- Calculer la volatilité réalisée (rolling std de log-returns)
- Construire une régression linéaire simple avec scikit-learn
- Visualiser la relation VIX / Vol. réalisée
- Interpréter les coefficients α, β et le R^2
- Analyser les résidus pour valider le modèle

Modules suivants — Pour aller plus loin

Module 2	Régression avec lags — VIX(t-1), VIX(t-5) comme prédicteurs
Module 3	Rolling window validation — éviter le look-ahead bias
Module 4	Features supplémentaires — rendement SPX, VIX change, skew
Module 5	Modèles non-linéaires — Ridge, Random Forest, XGBoost
Module 6	Backtesting d'une stratégie de trading sur la vol

■ ***Question d'approfondissement :*** *Le R^2 obtenu est calculé sur les mêmes données qu'on a utilisées pour entraîner le modèle (in-sample). Pourquoi est-ce problématique, et comment le corriger ? → Réfléchissez au concept de train/test split.*